

Advanced Natural Language Processing

Deep Learning: XAI for NLP

Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes – Course year 2026-2027



Maastricht University

XAI for AI in NLP



Maastricht University

Why Explanations

- **Risk and trust.** A model can be right for the wrong reason: shortcut learning, the Clever Hans effect from the BERT lecture. Explanations help us detect bias, spurious cues and unfaithful reasoning before they cause harm, especially in high-stakes domains such as health, law and hiring.
- **Regulation and accountability.** The EU AI Act and GDPR require transparency and, for affected persons, a meaningful explanation of decisions. Without explanations we cannot show compliance, audit a system, or assign responsibility when it fails.
- **Engineering priorities.** Explanations tell us *why* a model fails, so we fix the right thing: data, labels, prompt, retrieval or model. That saves time and money compared with trial-and-error retraining.

Explanations are not a nice-to-have. They are how we manage risk, satisfy the law and decide where to spend engineering effort. Which of the three matters most depends on who is asking: the user, the regulator or the developer.

Regulations in EU

- **Timeline (as of Sept 2026).** GPAI (general-purpose AI) model obligations, such as technical documentation, have applied since 2 Aug 2025. The Digital Omnibus entered into force on 27 Jul 2026 and moved Annex III high-risk duties (employment, credit, education) to 2 Dec 2027. Annex I product-embedded systems move to 2 Aug 2028. Article 50 transparency duties stayed on schedule, and only the labelling of synthetic content for systems already on the market got a grace period to 2 Dec 2026.
- **Right to explanation, AI Act (Art. 86).** A person adversely affected by a decision that a deployer takes on the basis of a high-risk system's output can request a clear and meaningful explanation. It covers the role of the AI system and the main elements of the decision. Art. 13 also requires high-risk systems to be transparent enough for deployers to interpret their output.
- **Right to explanation, GDPR.** Art. 22 restricts solely automated decisions with significant effects, and Arts. 13–15 give a right to "meaningful information about the logic involved". The "right to explanation" in Recital 71 is non-binding, so the exact scope has been debated and shaped by case law.

Regulation (EU) 2024/1689 (AI Act), Arts. 13, 50, 53 and 86

Regulation (EU) 2016/679 (GDPR), Arts. 13–15 and 22, Recital 71

Regulation (EU) 2026/1744 (Digital Omnibus on AI)

The law asks for a meaningful explanation for the affected person, not proof of the model's true internal mechanism. That is plausibility, not faithfulness. A LIME or SHAP output may satisfy the legal wording while saying little about what the model actually did, which is why the faithfulness question from the previous slide matters for compliance too.

- Transparency is given “if the processes that extract model parameters from training data and generate labels from testing data can be described and motivated by the approach designer”.
- Interpretability describes the possibility to comprehend the ML model and to present the underlying basis for decision-making in a way that is understandable to humans.
- Explainability is a concept that is recognized as important, but a joint definition is not yet available. It is suggested that explainability in ML can be considered as “the collection of features of the interpretable domain, that have contributed for a given example to produce a decision (e.g., classification or regression)”.

Faithfulness vs. plausibility: does the explanation reflect the model, or just convince us?

- **Two different questions (Jacovi & Goldberg, 2020).** *Faithfulness* asks whether an explanation accurately reflects the reasoning process the model actually used. *Plausibility* asks whether it is convincing to a human. An explanation can be plausible but unfaithful, and the reverse.
- **Why they come apart.** Plausibility is easy to optimize, and it is exactly what a human eye rewards: LIME's simplified linear surrogate, attention heatmaps, and chain-of-thought text can all look right without being the real mechanism. Faithfulness has to be tested, not assumed.
- **How to measure faithfulness: remove or keep the evidence.** Let r be the tokens the explanation marks as important, and let p be the model's probability for its original prediction.
 - *Comprehensiveness* = $p(x) - p(x \text{ without } r)$. Removing the important tokens should hurt, so a large drop is good.
 - *Sufficiency* = $p(x) - p(r \text{ only})$. The important tokens alone should be enough, so a small drop is good.

Never accept an explanation because it looks reasonable. Ask whether it is faithful to the model and test it by intervening on the input, because "convincing to humans" and "true to the model" are different properties.

The Psychology of explanation? What is a valid explanation?

Explanation are part of a social interaction:

- Valid explanations should be sound, cogent, convincing and create trust.
- The receiving party should be able to understand it.
- Step by step transfer of knowledge.

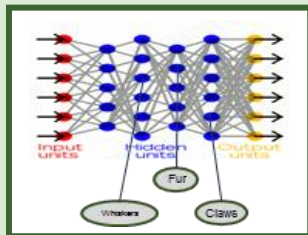
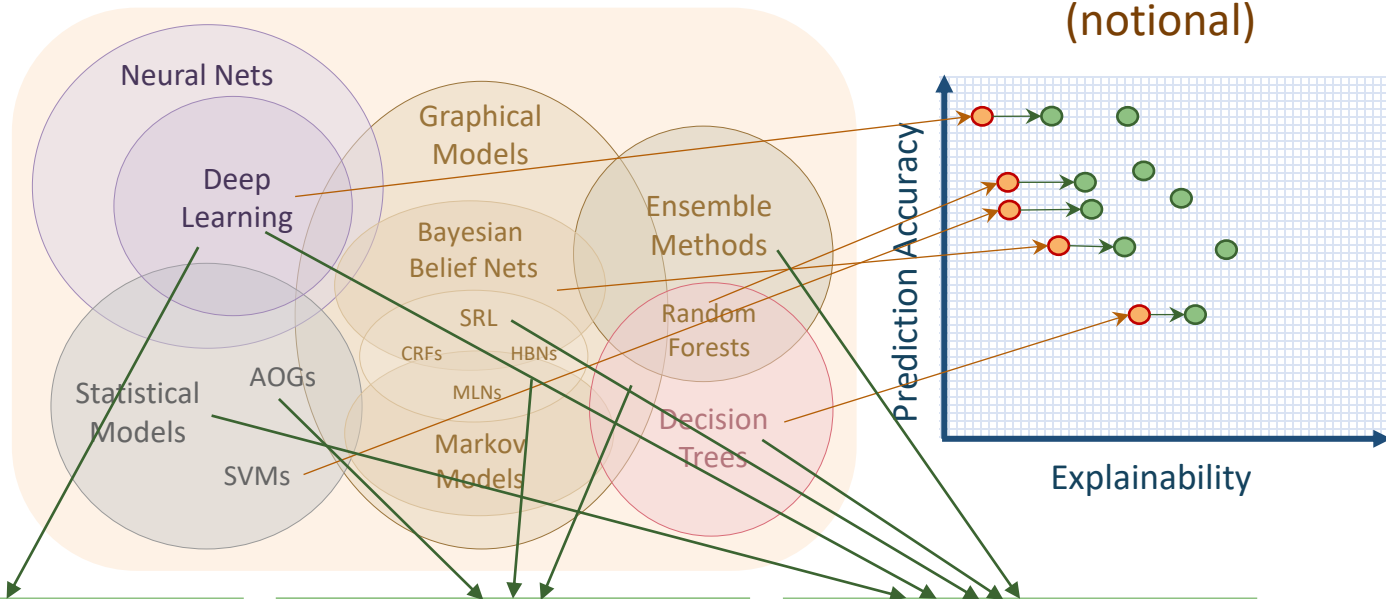
Moreover explanations:

- Must show the difference between possible outcomes of decision.
- Should be relevant.
- Can be incomplete. Emphasizing a few examples may be sufficient. No need to be overly complete.
- Should be believable;



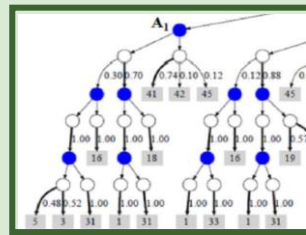
Learning Techniques (today)

Explainability (notional)



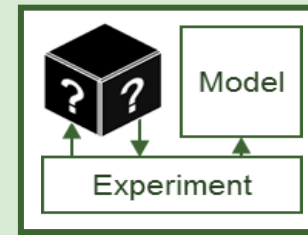
Deep Explanation

Modified deep learning techniques to learn explainable features



Interpretable Models

Techniques to learn more structured, interpretable, causal models



Model Induction

Techniques to infer an explainable model from any model as a black box

Source: Darpa

Large Language Models: We need to Understand them

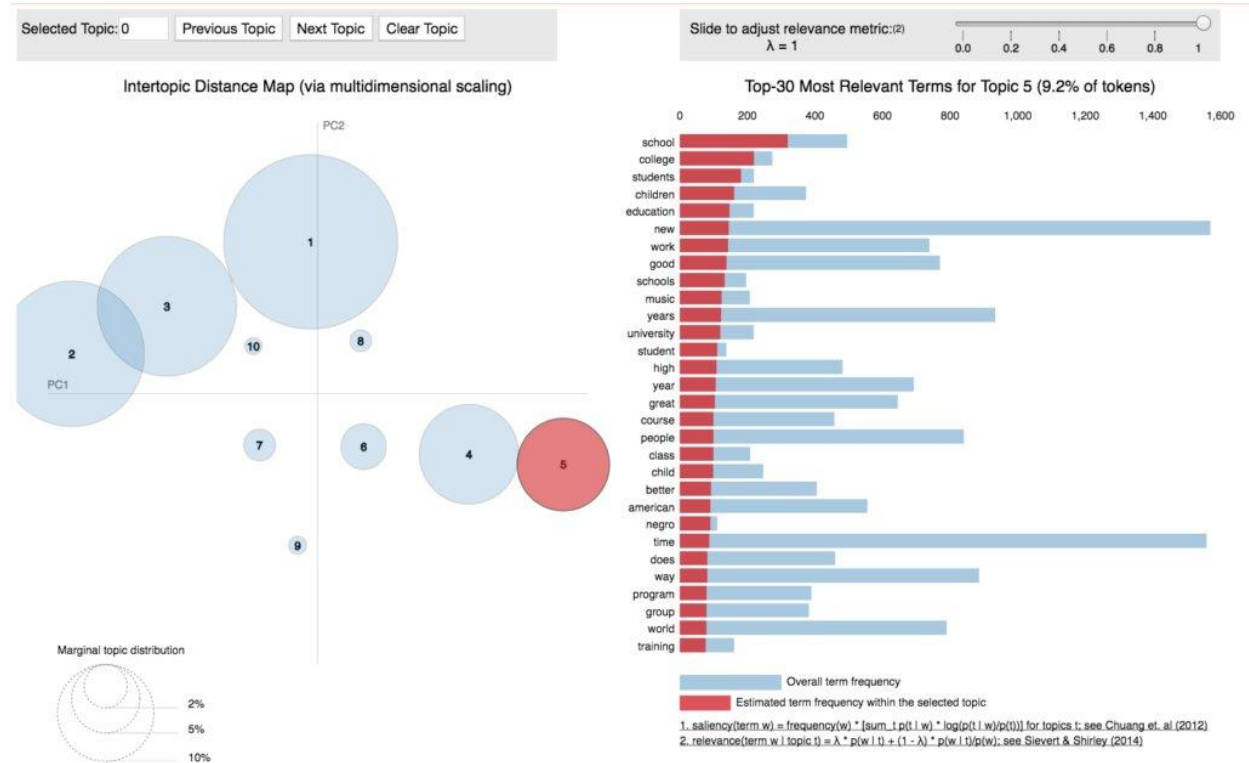
- Language models, which derive their power from colossal amounts of language data.
- “Organizations such as Google, Microsoft, and OpenAI expend extraordinary capital to build these models (sometimes millions of dollars for a single model), which then power products that impact billions of people.”
- We need to know what this technology can and can’t do, what risks it poses, so that we can both have a deeper scientific understanding and a more comprehensive account of its societal impact. Transparency is the vital first step towards these two goals.
- Large language models are general-purpose text interfaces that could be applied across a vast expanse of scenarios from question answering to summarization to toxicity detection. And for each use case, we have a broad set of desiderata: models should be accurate, robust, fair, efficient, and so on.
 - Broad coverage and recognition of incompleteness.
 - Multi-metric measurement to measure various aspects of such models.
 - Standardization of measurements.



Stanford University
Human-Centered
Artificial Intelligence

pyLDAViz for LDA and NMF

- Python library for interactive topic model visualization.
- Also has implementations for NMF.
- Shows topics and dominant keywords per topic.
- Standard Python implementations.



Local interpretable model-agnostic explanations (LIME)

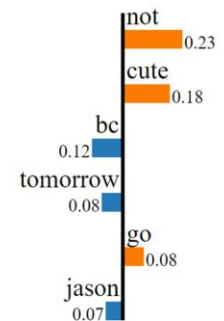
- Independent of training data. Consider ML as a black box model with input data points and predictions.
- Goal: to understand why the machine learning model made a certain prediction.
- LIME tests what happens to the predictions when you give variations of your data into the machine learning model.
- LIME generates a new dataset consisting of perturbed samples and the corresponding predictions of the black box model.
- On this new dataset LIME then trains an interpretable model, which is weighted by the proximity of the sampled instances to the instance of interest.
- Works with all types of ML

Prediction probabilities



none

negative



Text with highlighted words

about to go shopping again tomorrow bc the dress i got for jason aldean is not cute

Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin. "Why should I trust you?: Explaining the predictions of any classifier." Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining. ACM (2016)

SHAPLEY Values (SHAP)

$$\phi_i(v) = \frac{1}{|N|!} \sum_R [v(P_i^R \cup \{i\}) - v(P_i^R)]$$

- $\phi_i(v)$: the Shapley value for a specific player
- N : the number of players
- $v(P \cup \{i\})$: contribution of the set of players in order on the outcome, including player i
- $v(P)$: contribution of the set of players in order on the outcome (without player i)
- R : the permutation of order of the players (there are $N!$ permutations)

See also: Cite Lundberg & Lee (NeurIPS 2017), Winter, E. The Shapley value. Handbook of game theory with economic applications, 3:2025–2054, 2002, and here for a more approachable reference:

https://medium.com/@kalia_65609/interpreting-an-nlp-model-with-lime-and-shap-834ccfa124e4 or here:

<https://towardsdatascience.com/explainable-ai-xai-with-shap-regression-problem-b2d63fdca670>



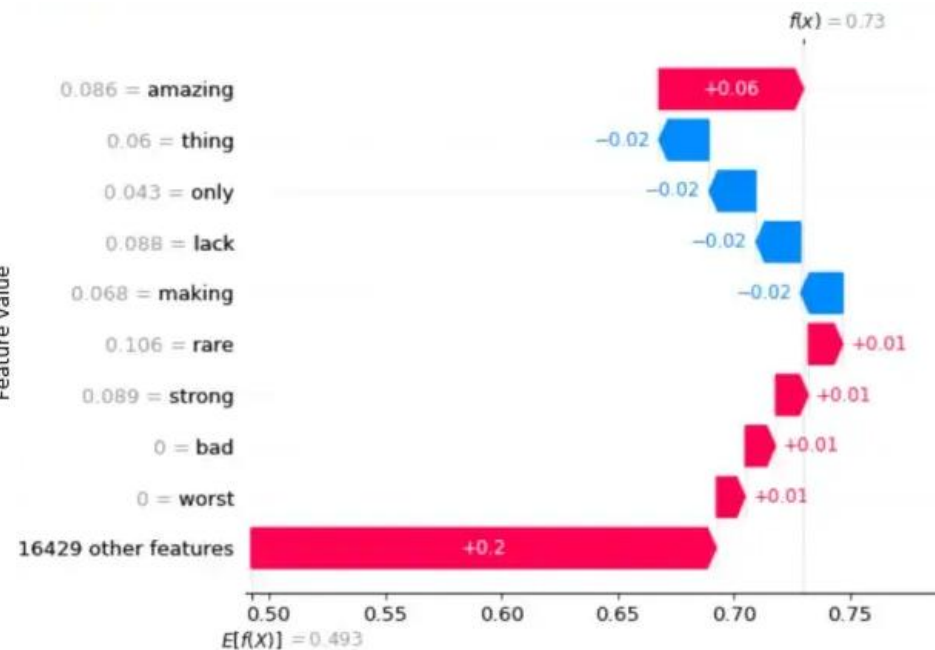
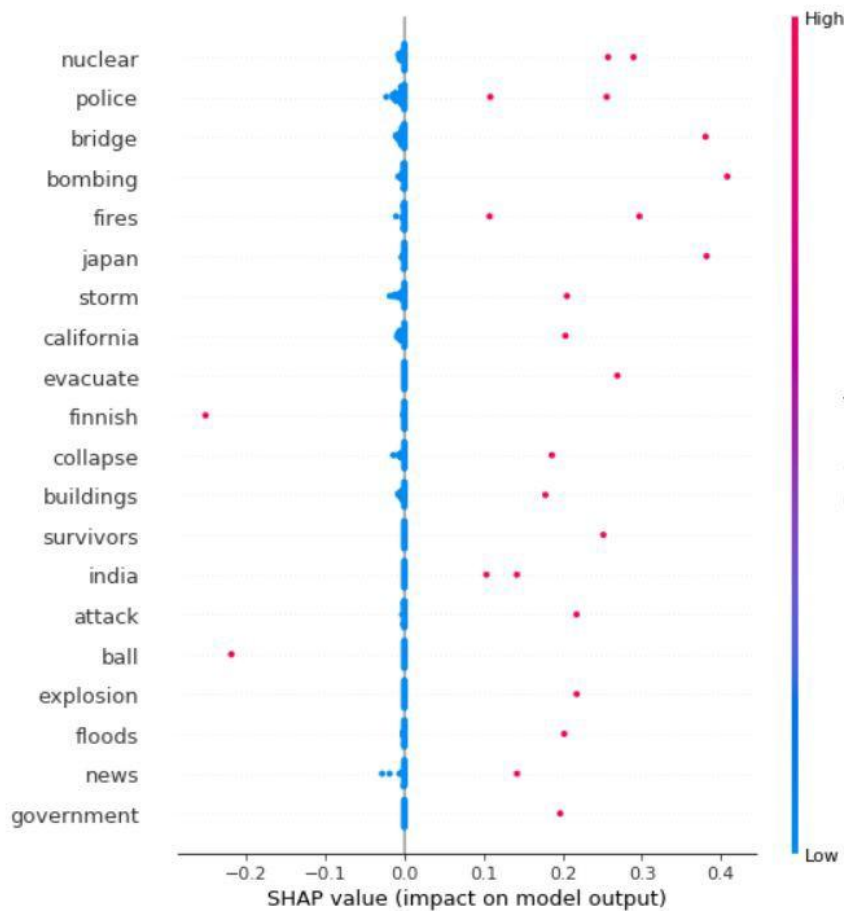
Chef	Steps Completed
C1	15
C2	20
C3	5
C1 U C2	35
C1 U C3	25
C2 U C3	30
C1 U C2 U C3	50

We then calculate the marginal contribution of each chef depending on the order in which they add their contributions, based on the table above. And, finally, calculate their mean contribution by dividing the sum of all their marginal contributions by the number of permutations. This results in the Shapley value:

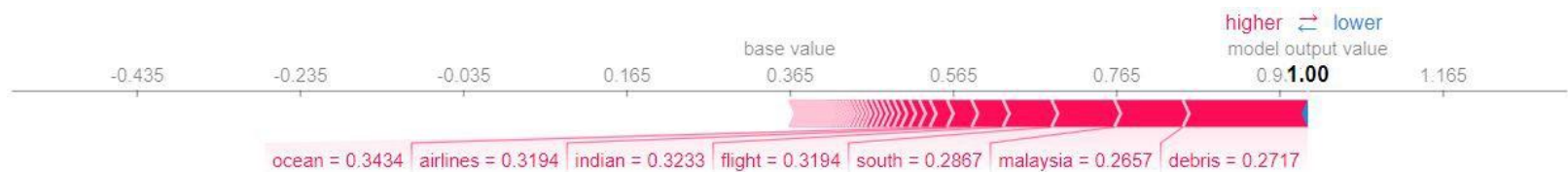
Table 3. Table showing the number of steps each chef

Order	C1 Contribution	C2 Contribution	C3 Contribution
C1, C2, C3	$v(C1) = 15$	$v(C1, C2) - v(C1) = 35 - 15 = 20$	$v(C1, C2, C3) - v(C1, C2) = 50 - 35 = 15$
C1, C3, C2	$v(C1) = 15$	$v(C1, C3, C2) - v(C1, C3) = 50 - 25 = 25$	$v(C1, C3) - v(C1) = 25 - 15 = 10$
C2, C3, C1	$v(C2, C3, C1) - v(C2, C3) = 50 - 30 = 20$	$v(C2) = 20$	$v(C2, C3) - v(C2) = 30 - 20 = 10$
C2, C1, C3	$v(C2, C1) - v(C2) = 35 - 20 = 15$	$v(C2) = 20$	$v(C2, C1, C3) - v(C2, C1) = 50 - 35 = 15$
C3, C1, C2	$v(C3, C1) - v(C3) = 25 - 5 = 20$	$v(C3, C1, C2) - v(C3, C1) = 50 - 25 = 25$	$v(C3) = 5$
C3, C2, C1	$v(C3, C2, C1) - v(C3, C2) = 50 - 30 = 20$	$v(C3, C2) - v(C3) = 30 - 5 = 25$	$v(C3) = 5$
Mean contribution	$1/6(15 + 15 + 20 + 15 + 20 + 20)$	$1/6(20 + 25 + 20 + 20 + 25 + 25)$	$1/6(15 + 10 + 10 + 15 + 5 + 5)$
Shapley Value	17.5	22.5	10

Table 4. Calculation of the Shapley value using the marginal contributions of each of the chefs depending on the order in which they contribute.

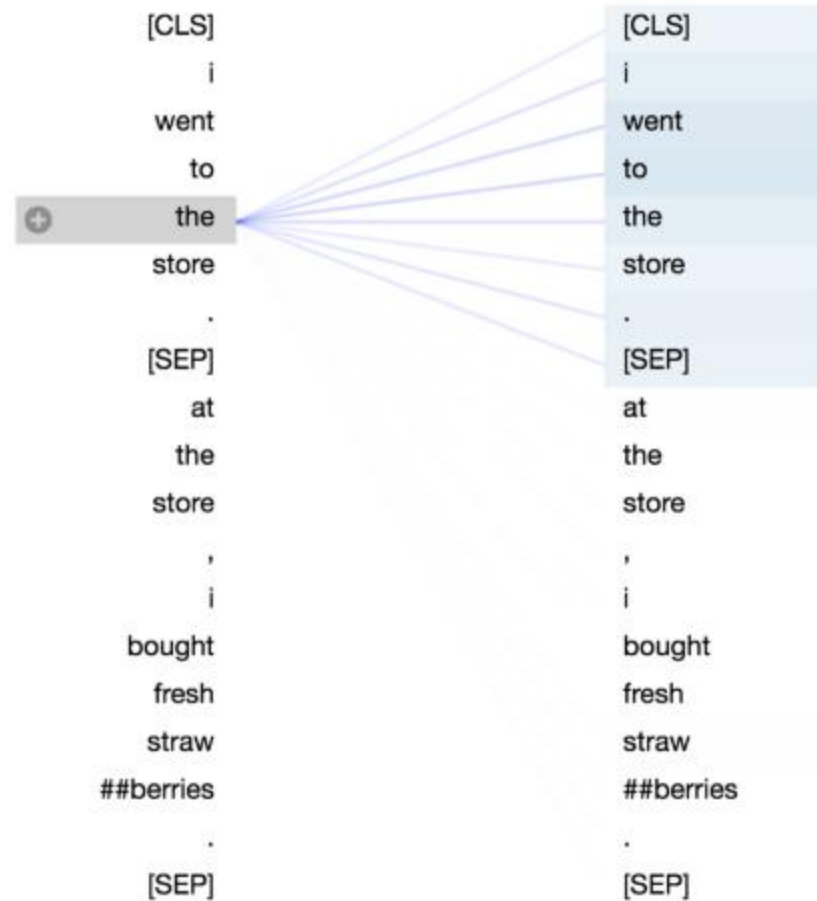


Source: <https://medium.com/nlplanet/two-minutes-nlp-explain-predictions-with-shap-values-2a0e34219177>

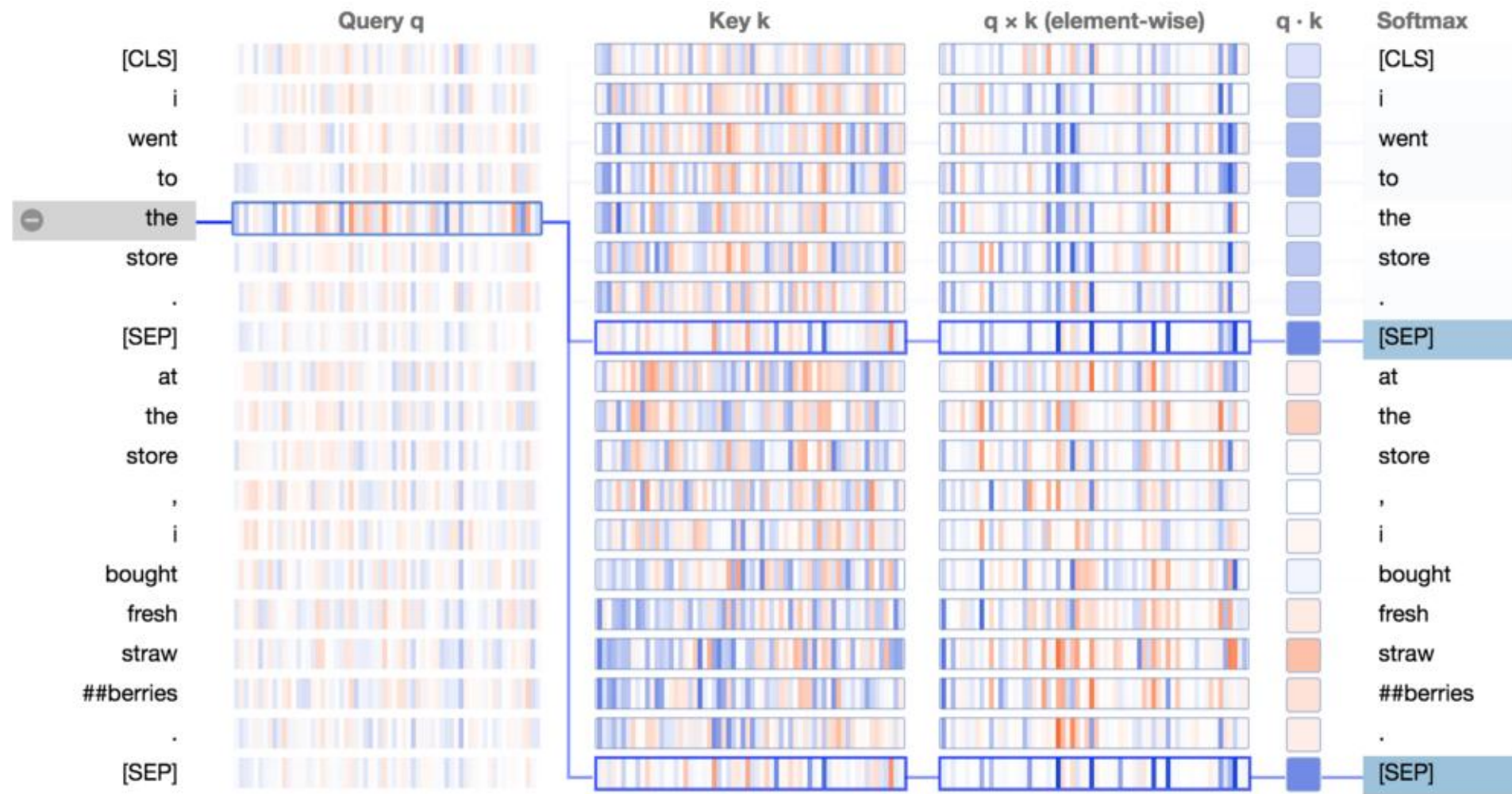


See also: Lundberg, S. (2018). SHAP. MIT. [Python package]. Retrieved from <https://github.com/slundberg/shap>

BERTViz: Head View

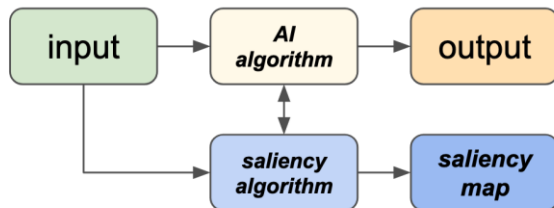


BERTViz: Neuron View



Saliency

- Saliency maps are heat maps that highlight features in the input to an AI model that were significant in the AI model's output predictions.
- Originates from computer vision with deep learning.



(A) Both models produce similar high-confidence detections (shown with red bounding boxes). **(B)** Despite having similar detections, the saliency maps corresponding to these detections reveal subtle differences in the input features used by the two models, e.g. TPH-YOLOv5 (top) focuses on the head and feet of pedestrians, while CenterNet (bottom) focuses on the torso.



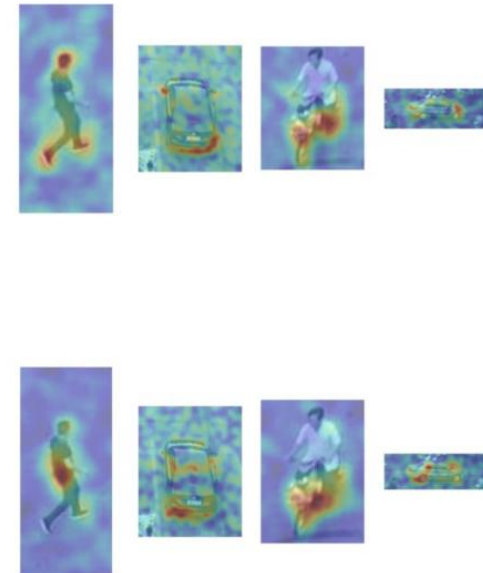
A
TPH-YOLOv5



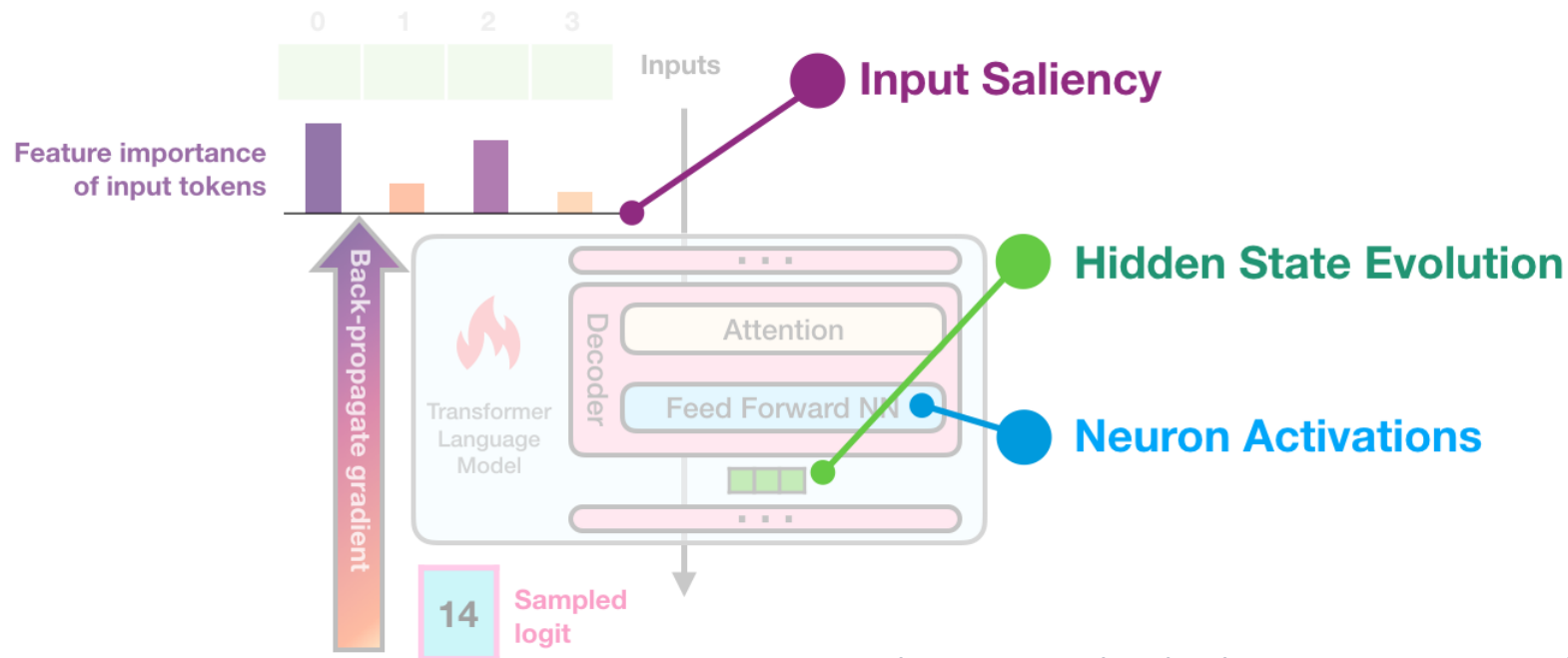
B
CenterNet



B



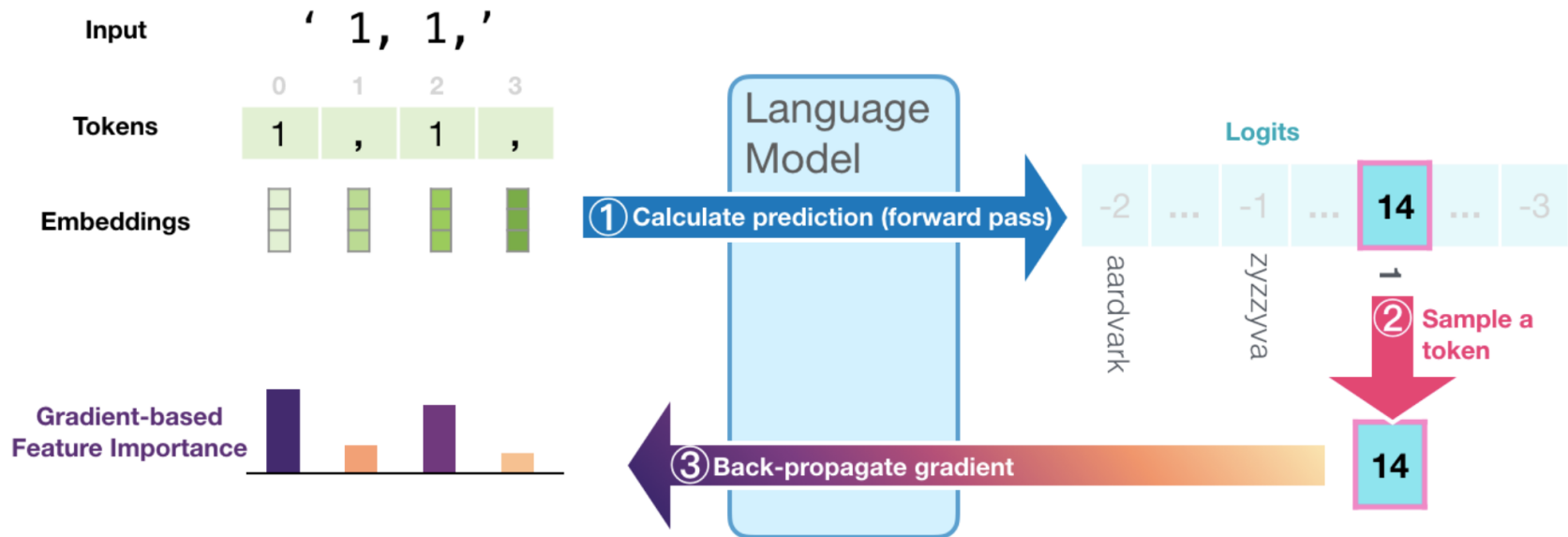
Saliency Methods for Neural Language Models



Source: <https://jalammar.github.io/explaining-transformers/>

- Input Saliency methods that score input tokens importance to the output. Can be used for classification as well as for generation.
- Neuron Activations and how individual and groups of model neurons spike in response to inputs and to produce outputs.
- Hidden State Evolution across the layers of the model and what it may tell us about each layer's role.

Gradient-based Saliency



1. The language model's final hidden state is projected into the model's vocabulary resulting in a numeric score for each token in the model's vocabulary. Using a softmax operation results in a probability score for each token.
2. We proceed to select a token (e.g. select the highest-probability scoring token or another strategy as we discussed).
3. By calculating the gradient of the selected logit (before the softmax) with respect to the inputs by back-propagating it all the way back to the input tokens, we get a signal of how important each token was in the calculation resulting in this generated token. That assumption is based on the idea that the smallest change in the input token with the highest feature-importance value makes a large change in what the resulting output of the model would be.

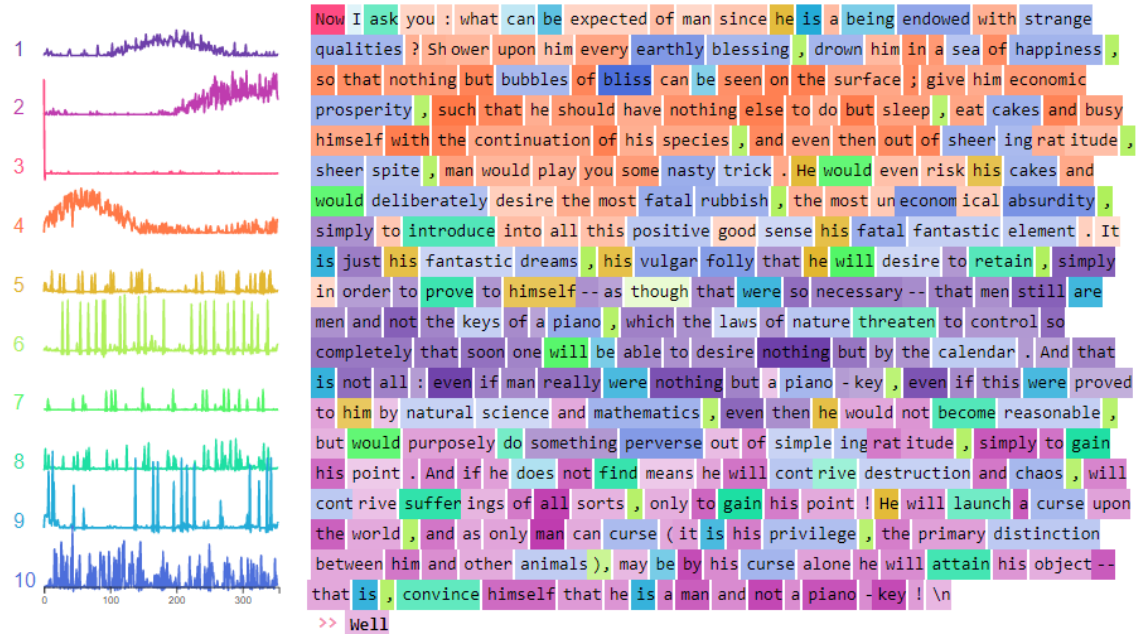
Non-Negative Matrix Factorization (NMF)

- Non-Negative Matrix Factorization (NMF) is a math technique that splits a large grid of positive numbers into two smaller positive grids that, when multiplied together, recreate the original data.
- Think of it like a recipe: it takes a finished dish (your main data) and breaks it back down into the individual ingredients (the building blocks) and the amounts used (the weights).
- We discuss this in detail in the course Agentic Conversational Search.

$$\begin{bmatrix} & \\ & \\ & \\ & \end{bmatrix}^W \times \begin{bmatrix} & & & & & \\ & & & & & \end{bmatrix}^H \approx \begin{bmatrix} & & & & & \\ & & & & & \\ & & & & & \\ & & & & & \end{bmatrix}^V$$

Neuron Activations

- Feed Forward Neural Network (FFNN) is one of the main components of a transformer on top of the self-attention.
- It contains 66% of all parameters, so it holds a significant part of the model's capacity.
- What happens where?
- Using Non-negative Matrix Factorization (NMF) for reducing the dimensionality into groups that are potentially individually more interpretable.



Explorable: Ten Factors From The Underground

Tap or hover over the sparklines on the left to isolate a certain factor

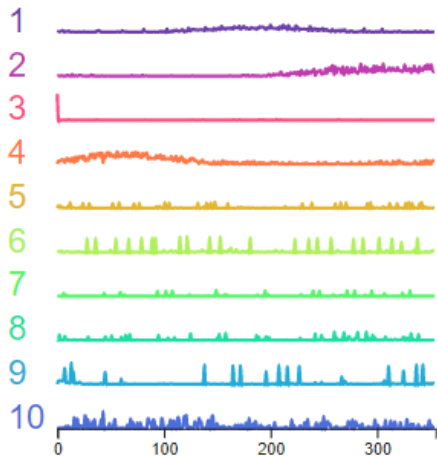
Ten Factors from the activations the neurons in Layer 0 in response to a passage from Notes from Underground by Dostoevsky.

- Factors that focus on specific portions of the text (beginning, middle, and end). This is interesting as the only signal the model gets about how these tokens are connected are the positional encodings. This indicates the neurons that are keeping track of the order of words. Further examination is required to assess whether these FFNN neurons directly respond to the time signal, or if they are responding to a specific trigger in the self-attention layer's transformation of the hidden state.
- Factors corresponding to linguistic features, for example, we can see factors for pronouns (he, him), auxiliary verbs (would, will), other verbs (introduce, prove -- notice the understandable misfiring at the 'suffer' token which is a partial token of 'sufferings', a noun), linking verbs (is, are, were), and a factor that favors nouns and their adjectives ("fatal rubbish", "vulgar folly").
- A factor corresponding to commas, a syntactic feature.
- The first-token factor

Go here: for a live view of what happens in the different layers: <https://jalammar.github.io/explaining-transformers/>

Try this all yourself with ECCO

```
[ ] nmf_2.explore()
```



Now I ask you : what can be expected of man since he is a being endowed with strange qualities ? Shower upon him every earthly blessing , drown him in a sea of happiness , so that nothing but bubbles of bliss can be seen on the surface ; give him economic prosperity , such that he should have nothing else to do but sleep , eat cakes and busy himself with the continuation of his species , and even then out of sheer ingratitude , sheer spite , man would play you some nasty trick . He would even risk his cakes and would deliberately desire the most fatal rubbish , the most uneconomical absurdity , simply to introduce into all this positive good sense his fatal fantastic element . It is just his fantastic dreams , his vulgar folly that he will desire to retain , simply in order to prove to himself -- as though that were so necessary -- that men still are men and not the keys of a piano , which the laws of nature threaten to control so completely that soon one will be able to desire nothing but by the calendar . And that is not all : even if man really were nothing but a piano - key , even if this were proved to him by natural science and mathematics , even then he would not become reasonable , but would purposely do something perverse out of simple ingratitude , simply to gain his point . And if he does not find means he will contrive destruction and chaos , will contrive sufferings of all sorts , only to gain his point ! He will launch a curse upon the world , and as only man can curse (it is his privilege , the primary distinction between him and other animals) , may be by his curse alone he will attain his object -- that is , convince himself that he is a man and not a piano - key ! \n

>> This

- https://colab.research.google.com/github/jalammar/ecco/blob/main/notebooks/Ecco_Neuron_Factors.ipynb#scrollTo=CLDsArU2N0lr



Jacobian Matrix & J-Space

- **Jacobian matrix:** In calculus, a Jacobian matrix is the matrix of all first-order partial derivatives of a vector-valued function. It describes how small changes in a system's inputs translate into changes in its outputs.
- **Application to Claude:** The J-Lens asks, for any internal activation inside the model, “if this activation were nudged slightly, how would that change the probability that the model outputs each possible word later on?” This sensitivity relationship is mathematically a Jacobian — hence the name “Jacobian Lens.”
- **How it is computed:** For every layer ℓ of Claude's residual stream, the J-Lens computes the averaged linearized effect of an activation on the model's final output logits: formally, $J\ell = E[\partial h_{\text{final}} / \partial h\ell]$, averaged over roughly 1,000 diverse prompts and token positions. Applying this averaged Jacobian to any activation at any layer produces a ranked list of vocabulary words — the words the model is internally “oriented toward” at that moment, whether or not it ever says them. The paper reports that a typical J-Space holds only 10–25 active J-Lens vectors at once, together accounting for less than 10% of a given representation's activation variance.

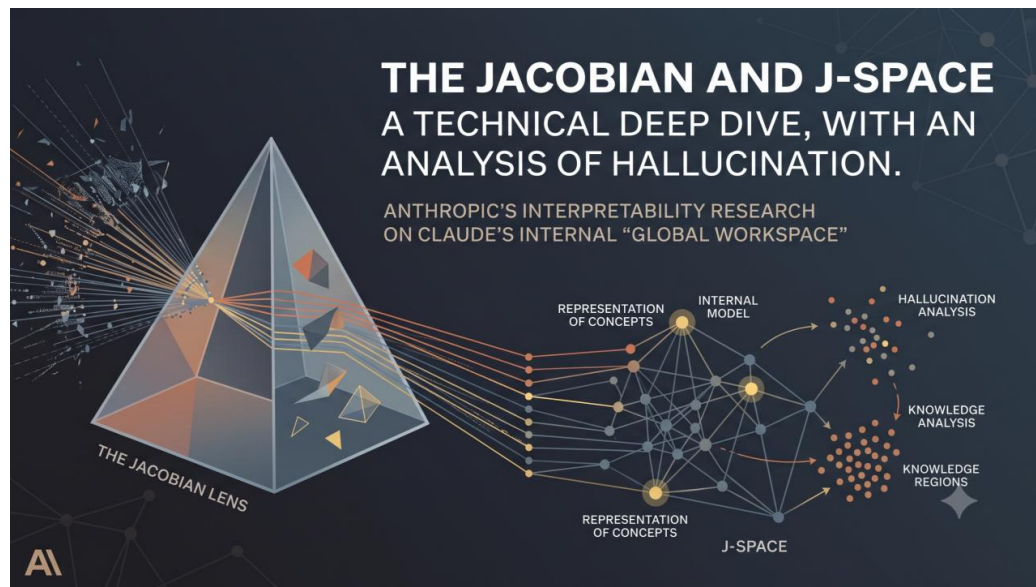
Example: When Claude reads a snippet of code containing a bug nobody has flagged, applying the J-Lens to its internal state reveals the token “ERROR” rising to the top of the list — even though Claude's visible output says nothing about a bug yet.

Recent Development: The Jacobian Lens and “J-Space” (2026)

- The problem: the Logit Lens (applying the final unembedding matrix to a middle layer) ignores the non-linear transformations still to come, giving an inaccurate read of what a middle layer “means”.
- The fix: Anthropic’s Jacobian lens (J-lens) computes the Jacobian of the remaining layers with respect to a middle layer — a calculus-based “fast-forward” that projects hidden activations into vocabulary space far more accurately than the Logit Lens.
- J-space: applying the J-lens to Claude Opus 4.6 revealed a privileged internal workspace holding concepts, multi-step reasoning, and even hidden intentions (e.g. recognizing it is being tested) — running in parallel to, and structurally separate from, the track that generates the visible output.

Why it belongs here: a direct, gradient-based successor to today’s saliency and hidden-state methods — concrete evidence that attention weights alone cannot explain what a model is “thinking”.

Gurnee, Lindsey et al., “Verbalizable Representations Form a Global Workspace in Language Models”, transformer-circuits.pub, July 2026. See also: [MIT Technology Review coverage](#)



Questions



Maastricht University

Exercise 1: Model-Agnostic Post-Hoc Interpretability (LIME vs. SHAP)

The slides examine post-hoc local explanation techniques that treat the NLP model as a black box. Consider a sentiment classification model predicting whether online product reviews are negative or non-negative.

- Explain how **LIME** (Local Interpretable Model-agnostic Explanations) creates local explanations for a single review. How does it perturb input text, obtain predictions from the black-box model, and fit an interpretable surrogate model weighted by proximity?

In the lecture, **Shapley values** from cooperative game theory:

- $\phi_i(v) = \frac{1}{|N|!} \sum_R [v(P_i^R \cup \{i\}) - v(P_i^R)]$
- Using the 3-chef analogy from slides 9–10, explain what $v(P_i^R \cup \{i\}) - v(P_i^R)$ represents for a specific feature (or token) i . Why does calculating exact Shapley values require computing all $|N|!$ permutations of features, and why does this become computationally prohibitive for long textual sequences?
- The example sentence *"about to go shopping again tomorrow bc the dress i got for jason aldean is not cute"* receives a negative prediction driven by positive weights on *"not"* and *"cute"*. What is the primary limitation of token-attribution methods when explaining non-linear semantic interactions like negation or sarcasm?

Exercise 2: White-Box Interpretability (Attention Visualizations vs. Gradient Saliency)

The slides contrast visual inspection of internal attention weights (BERTViz) with gradient-based saliency methods.

- **BERTViz** distinguish between the **Head View** and the **Neuron View**. What specific information does the neuron view provide regarding the element-wise interaction ($q \times k$) and the subsequent softmax attention weight before multiplying by value vectors?
- We also illustrate **Gradient-based Saliency**. Explain how backpropagating the gradient of the chosen unnormalized logit score (before softmax) back to the input token embeddings provides a feature importance score. What core mathematical assumption connects the magnitude of $\frac{\partial \text{logit}}{\partial x_i}$ to the impact of that token on the final output?
- Based on the debate cited in earlier lectures ("*Attention is not Explanation*" vs. "*Attention is not not Explanation*"), explain why inspecting raw attention matrices can produce misleading explanations of model decisions, whereas gradient-based saliency directly measures sensitivity to changes in the input representation.

Exercise 3: Internal Representations (Hidden State Evolution and Neuron Factorization)

The lecture slides outline techniques for peering inside Transformer layers beyond simple input-attribution maps, focusing on Feed-Forward Network (FFN) activations and hidden states.

- We show that the position-wise Feed-Forward Networks (FFNs) contain approximately 66% of all parameters in a standard Transformer block. Why does analyzing self-attention alone fail to capture the majority of the model's stored capacity and non-linear transformations?
- How does the **ECCO** framework utilize **Non-negative Matrix Factorization (NMF)** on neuron activations across hidden layers to extract interpretable latent factors? What do distinct activation factors reveal about how different layers process low-level syntax versus abstract semantic properties?
- Explain the concept of **Hidden State Evolution** across the depth of a decoder or encoder stack. If an output token's top logit changes dynamically from layer 1 to layer 12, what does this tell an engineer debugging a model failure (such as hallucination or shortcut learning)?

Discussion



On the "Faithfulness vs. Plausibility" Trap: Who Is the Explanation For?

In the psychology of XAI, we note that a valid human explanation must be sound, convincing, and build trust—often by highlighting just a few salient examples rather than complete mathematical truth.

Yet tools like LIME fit an intentionally simplified linear surrogate around a local data point to satisfy human intuition.

If a local explanation is designed primarily to look believable and cogent to a human observer, are we discovering how the high-dimensional neural network actually arrived at its decision, or are we merely generating a comforting post-hoc rationalization that human psychology finds acceptable?

At what point does optimizing for human interpretability turn into self-deception?

On Attention Maps vs. Gradient Saliency: Correlation vs. Causation

Tools like BERTViz let us inspect attention lines connecting words directly, tempting us to conclude that because token A pays 90% attention to token B , token B caused the model's output.

However, literature cited in earlier lectures warned that 'Attention is not Explanation', and in this lecture we see gradient-based saliency backpropagating the unnormalized logit to measure true input sensitivity.

If attention weights are merely intermediate routing coefficients that get multiplied by value vectors, fed through non-linear feed-forward layers containing 66% of the network's parameters, and blended via residual streams, can inspecting attention weights ever give us a causal explanation?

Why do researchers and developers keep relying on attention heatmaps if gradient saliency is mathematically closer to causal sensitivity?



On the Feed-Forward Blind Spot: Are We Looking Where the Light Is?

The lecture highlights that Transformer Feed-Forward Networks (FFNs) hold roughly two-thirds of all parameters in the architecture, acting as the primary store of model capacity and memory.

Yet the overwhelming majority of popular NLP explainability tools—such as BERTViz or self-attention heatmaps—focus exclusively on the multi-head self-attention blocks.

By concentrating our explainability research on attention graphs simply because they are easy to visualize as lines between words, are we committing the classic 'streetlight fallacy'—searching where it is easy to look rather than where the actual computation lives?

What must interpretability frameworks (like neuron activation factorization and hidden state evolution) accomplish before we can claim we understand what an LLM is doing?

